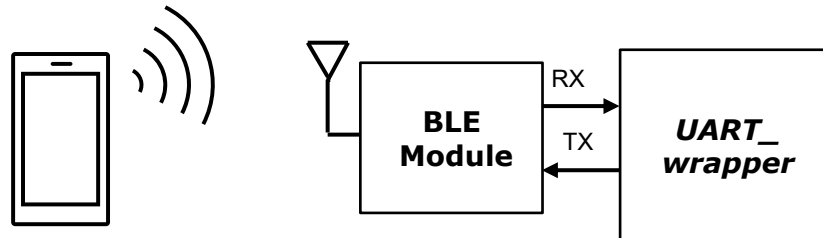




Exercise 14: UART Wrapper

- The MazeRunner receives 16-bit commands via Bluetooth
 - Commands control how it will move
 - BLE module provides data using UART
 - high byte first, then low byte
 - We will send an ack. response (1 byte) back to BLE module
- Since UART is byte based, we want a wrapper to package two bytes into a single 16-bit command

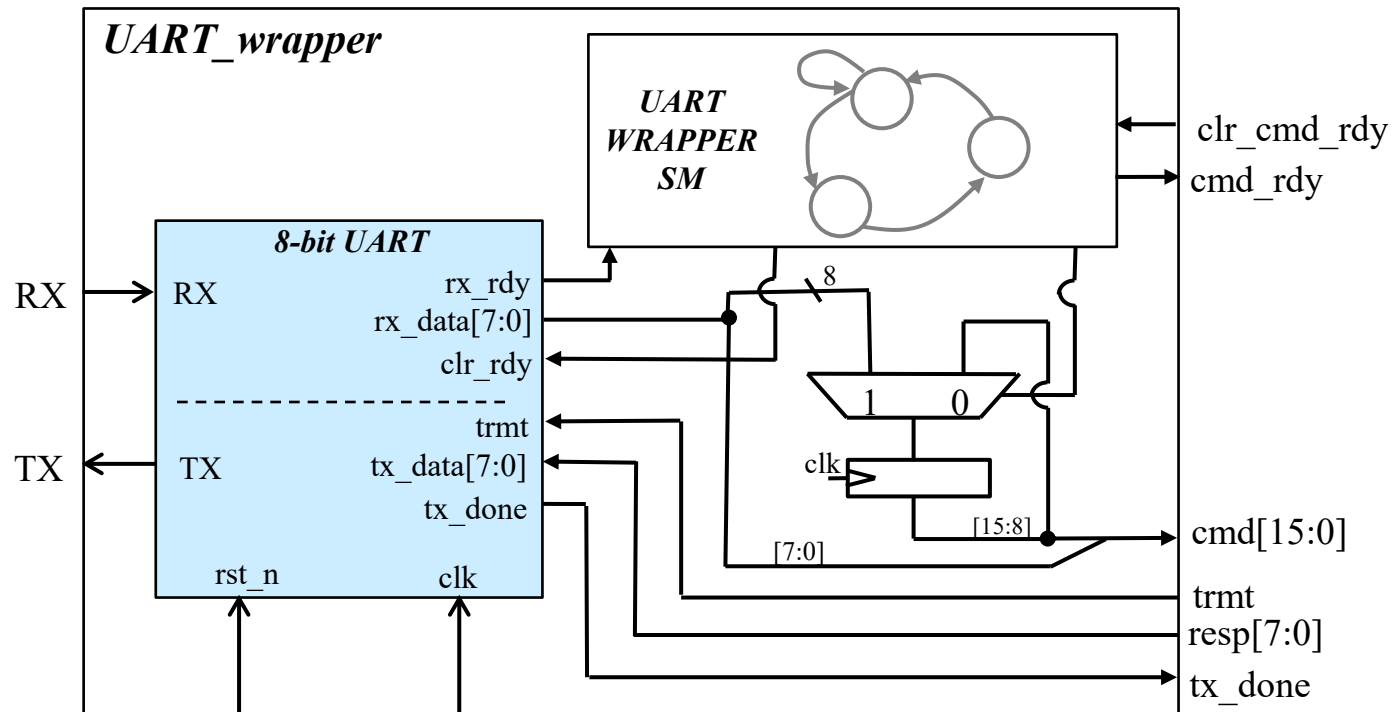


- Receiver side, we need:
 - Holding register
 - FSM to control data movement & output signals
- Transmitter side – simply pass signals on to UART_tx



Exercise 14: UART Wrapper

- **UART.sv** is provided – simply instantiates your **UART_tx** and **UART_rx**

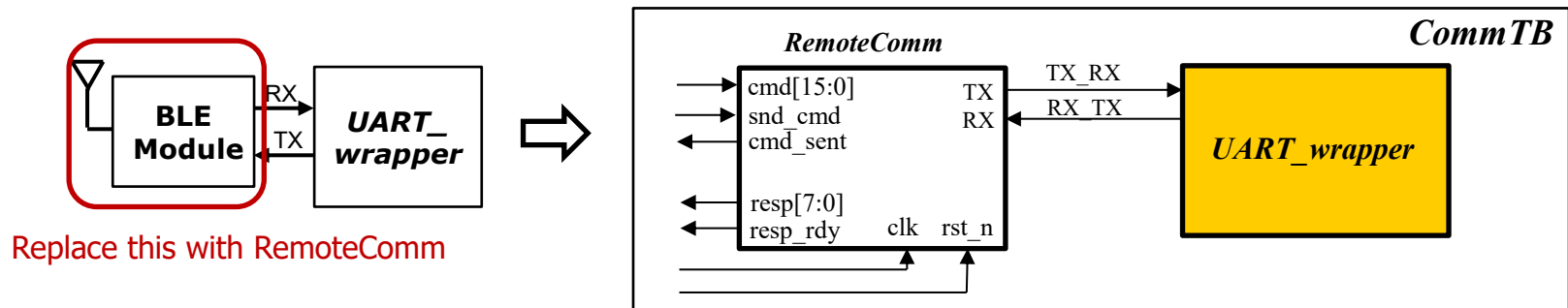


Signal	Dir	Description
clk, rst_n	in	50MHz clock & asynch active low reset
RX	in	Receive line from Bluetooth module (19200 baud)
cmd_rdy, cmd[15:0]	out	When cmd_rdy is asserted, cmd is the 16-bit command received
clr_cmd_rdy	in	Used to knock down cmd_rdy. Bookkeeping for consumer.
trmt & resp[7:0]	in	resp is sent to Bluetooth module upon a pulse on trmt
tx_done	out	Asserted when resp has been sent
TX	out	Transmit line to Bluetooth module (19200 baud)



Exercise 14: UART Wrapper Testing

- We will build a **RemoteComm** module that mimics what the BLE module would do
 - Take a 16-bit command & split it into 2x 8-bit UART transmissions
 - No special handling of Ack response – just connect directly to UART_rx
 - Essentially reverse of UART_wrapper



- Create **RemoteComm.sv**
- Create a self-checking testbench (**CommTB.sv**) for **UART_wrapper.sv** using **RemoteComm**



RemoteComm Details

- Instantiates UART transceiver and holding register for cmd
- Create FSM to send 2 bytes
- Set cmd_snt signal when entire transmission is complete

